

VOLUME 1 • B.TECH CSE STUDY SERIES

C PROGRAMMING

Complete Professional Handwritten Notes

*From Zero Prerequisites to Advanced Systems Programming • Pointers,
Memory Architecture, Structures, File I/O, Viva & Technical Interview
Prep*

```
#include <stdio.h>
int main() {
    printf("Hello CSE!")
    return 0; }
```

✓ Complete 13-Part Topic Structure

✓ Pointer & Memory Diagrams

✓ 30+ Exam & Viva Q&A

✓ 30+ Tech Interview Q&A

✓ 4 Real-World Projects

✓ 1-Day Exam Cheat Sheet

Designed for University Exams (B.Tech/BCA/MCA) & GATE / Product Company Placements



TABLE OF CONTENTS — C PROGRAMMING

Ch 1. C Language Fundamentals & Compilation Flow	Page 02
Ch 2. Variables, Data Types & Memory Storage	Page 05
Ch 3. Operators, Expressions & Input/Output	Page 08
Ch 4. Control Flow: Conditionals & Loops	Page 12
Ch 5. Functions, Recursion & Stack Frames	Page 17
Ch 6. Arrays, Matrices & String Operations	Page 22
Ch 7. Pointers, Memory Architecture & Pointer Arithmetic	Page 28
Ch 8. Dynamic Memory Allocation (Heap Management)	Page 34
Ch 9. Structures, Padding, Unions & Enums	Page 39
Ch 10. File Handling & Command-Line Arguments	Page 45
Ch 11. Comparison Tables & Visual Mind Maps	Page 50
Ch 12. Real-World Practical Projects	Page 54
Ch 13. Practice Problems (Basic → Advanced)	Page 60
Ch 14. 30+ University Viva Questions & Answers	Page 64
Ch 15. 30+ Technical Interview Questions & Answers	Page 69
Ch 16. 1-Day Revision & Last-Minute Exam Cheat Sheet	Page 74



C PROGRAMMING — VISUAL MIND MAP

C LANGUAGE ARCHITECTURE & ROADMAP

1. Basics & Memory

Source Code → Preprocessor → Compiler → Assembler → Linker → Executable

Primitive Types (int, float, char, double)

Storage Classes (auto, register, static, extern)

2. Control Flow & Modular Logic

If-Else / Switch

Loops (For, While, Do-While)

Functions & Call Stack Frames

Recursion & Base Cases

3. Memory & Pointers (Core Systems C)

Pointers & Addresses (&, *)

Pointer Arithmetic

Array-Pointer Equivalence

Dynamic Allocation (malloc, calloc, realloc, free)

4. Data Structures & I/O

Structures (struct) & Padding

Unions & Memory Overlap

File Streams (fopen, fread, fwrite, fclose)

CHAPTER 1: C COMPILATION PROCESS & ARCHITECTURE

💡 1. Concept: The C Compilation Pipeline

1. What is it?

The process of transforming high-level C source code (.c) into machine-executable binary code (.exe / executable) through 4 distinct sequential stages.

2. Why do we need it?

Computers and CPU instruction sets cannot directly understand human-readable text syntax like `printf()`. The compilation process translates code into machine language (0s and 1s) tailored to the target CPU architecture.

3. How does it work? (Internal Stages)

Stage 1: Preprocessing (cpp) – Processes macros (`#define`), header expansion (`#include`), strips comments. Output: `file.i`

Stage 2: Compilation (gcc) – Translates preprocessed code into assembly instructions. Output: `file.s`

Stage 3: Assembly (as) – Converts assembly code into machine code object bytes. Output: `file.o` / `file.obj`

Stage 4: Linking (ld) – Combines object files with standard library compiled code (e.g. `printf` from `libc`) to form the final executable executable.

SVG VISUAL DIAGRAM: C COMPILATION STAGES



4. Syntax & Basic Program Skeleton

```
// First C Program
#include <stdio.h> // Preprocessor directive for standard I/O

int main() {      // Main entry point function
    printf("Welcome to B.Tech CSE C Notes!\n");
    return 0;     // Indicates successful execution (exit status 0)
}
```

5. Real-World Use

Used in Operating System kernels (Linux Kernel is written in C), embedded devices, device drivers, database engines (MySQL, PostgreSQL storage layer), and Python runtime interpreter (CPython).

 **Common Mistake:** Forgetting to return 0 or omitting `#include <stdio.h>` leading to implicit function declaration warnings in modern GCC compilers.

 **Exam / Viva Point:** In exams, always write `int main()` instead of `void main()` because standard C requires main to return an integer exit code to the host OS.

CHAPTER 2: POINTERS & MEMORY ARCHITECTURE



2. Concept: Pointers and Memory Addresses

1. What is a Pointer?

A pointer is a variable whose value is the memory address of another variable.

2. Why do we need Pointers?

Pointers enable direct memory manipulation, dynamic allocation on Heap, efficient pass-by-reference in functions (avoiding large struct copies), and constructing data structures like Linked Lists and Trees.

VISUALIZATION: POINTER & MEMORY ADDRESS MAP



4. Syntax & Code Example

```
#include <stdio.h>

int main() {
    int val = 100;
    int *ptr = &val; // ptr stores address of val

    printf("Value of val: %d\n", val);
    printf("Address of val (&val): %p\n", (void*)&val);
    printf("Value stored in ptr: %p\n", (void*)ptr);
    printf("Value dereferenced (*ptr): %d\n", *ptr);

    *ptr = 200; // Modifying val indirectly via pointer
    printf("Updated val: %d\n", val);
    return 0;
}
```

Output: Value of val: 100 Address of val (&val): 0x7ffeebf5ac Value stored in ptr: 0x7ffeebf5ac Value dereferenced (*ptr): 100 Updated val: 200

 **Technical Interview Question:** What is a *Dangling Pointer* vs a *Wild Pointer*?

- **Wild Pointer:** Uninitialized pointer containing garbage memory address.
- **Dangling Pointer:** Pointer pointing to a memory location that has been freed / deallocated.

CHAPTER 3: DYNAMIC MEMORY ALLOCATION (HEAP MANAGEMENT)



Dynamic Memory Functions: malloc, calloc, realloc, free

Function	Syntax	Initialization	Description
malloc()	(type*) malloc(size_in_bytes)	Garbage Values	Allocates single contiguous block of memory on Heap. Returns NULL on failure.
calloc()	(type*) calloc(num, size)	Zero Initialized	Allocates multiple contiguous blocks and initializes all bytes to 0.
realloc()	(type*) realloc(ptr, new_size)	Uninitialized expansion	Resizes previously allocated memory block without losing existing data.
free()	free(ptr); ptr = NULL;	Deallocated	Releases allocated Heap memory back to OS to prevent Memory Leaks.

CHAPTER II: COMPARISON TABLES (EXAM & VIVA REVISION)

1. Call by Value vs Call by Reference

Parameter	Call by Value	Call by Reference
Passing Mechanism	Passes copy of actual argument value.	Passes memory address (&) of argument.
Original Modification	Changes inside function do NOT affect original variable.	Changes inside function directly modify original variable.
Memory Efficiency	Less efficient for large structs (copies memory).	Highly efficient (passes single pointer address).

2. Structure vs Union

Feature	Structure (struct)	Union (union)
Memory Allocation	Allocates separate memory for every member.	Shares single memory block equal to largest member size.
Member Access	All members can be accessed simultaneously.	Only ONE member can hold a valid value at any given time.
Total Size	Sum of sizes of all members (+ padding).	Size of the largest member.

CHAPTER 12: REAL-WORLD PRACTICAL PROJECT



Project: Student Record Management System (File-Based)

Project Requirements: Build a command-line application in C that allows creating, listing, searching, and saving student records (ID, Name, GPA) into a binary file.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    int id;
    char name[50];
    float gpa;
} Student;

void addStudent() {
    FILE *fp = fopen("students.dat", "ab");
    if (!fp) { printf("Error opening file!\n"); return; }

    Student s;
    printf("Enter Student ID: "); scanf("%d", &s.id);
    printf("Enter Name: "); scanf("%[^
]s", s.name);
    printf("Enter GPA: "); scanf("%f", &s.gpa);

    fwrite(&s, sizeof(Student), 1, fp);
    fclose(fp);
    printf("✅ Record saved successfully!\n");
}

void displayStudents() {
    FILE *fp = fopen("students.dat", "rb");
    if (!fp) { printf("No records found!\n"); return; }

    Student s;
    printf("\n--- STUDENT RECORDS ---\n");
    while (fread(&s, sizeof(Student), 1, fp)) {
        printf("ID: %d | Name: %-20s | GPA: %.2f\n", s.id, s.name, s.gpa);
    }
    fclose(fp);
}

int main() {
    int choice;
    while (1) {
        printf("\n1. Add Student\n2. Display All Students\n3.
Exit\nChoice: ");
        if (scanf("%d", &choice) != 1 || choice == 3) break;
        if (choice == 1) addStudent();
        else if (choice == 2) displayStudents();
    }
    printf("Exiting program.\n");
}

```

```
return 0;  
}
```

CHAPTER 14: 30+ IMPORTANT UNIVERSITY VIVA QUESTIONS

Q1. What is the static keyword in C?

Ans: A static local variable retains its value between function calls and is stored in the Data Segment. A static global variable limits its scope to the file in which it is defined.

Q2. What is a NULL Pointer vs Void Pointer?

Ans: A NULL pointer points to nothing (address 0x0). A void* (generic pointer) can hold address of any data type but must be typecasted before dereferencing.

Q3. What is Memory Leak in C?

Ans: A memory leak occurs when memory allocated dynamically on Heap via `malloc()` is no longer needed but never released using `free()`.

CHAPTER 15: TECHNICAL INTERVIEW QUESTIONS

Q. Implement a function to reverse a string in-place using pointers.

```
void reverseString(char *str) {  
    if (!str) return;  
    char *start = str;  
    char *end = str + strlen(str) - 1;  
    while (start < end) {  
        char temp = *start;  
        *start++ = *end;  
        *end-- = temp;  
    }  
}
```

CHAPTER 16: 1-DAY REVISION & EXAM CHEAT SHEET

Quick Revision Summary

- **Data Type Sizes (64-bit OS):** char = 1B, short = 2B, int = 4B, long = 8B, float = 4B, double = 8B, pointer = 8B.
- **Operator Precedence:** Parentheses () > Unary ++, --, *ptr, & > Arithmetic *, /, % > Additive +, - > Relational > Logical > Assignment.
- **Pointer Rule:** arr[i] is equivalent to *(arr + i).
- **Struct Padding:** Members are aligned to multiples of their natural size (e.g. 4-byte int aligned at 4-byte boundary).